TABLE OF CONTENTS

Executive Summary ii
List of Tables iii
List of Figures iv
List of Abbreviations v
List of Symbols vi

1 INTRODUCTION 1

1.1 Background 1
1.1.1 Security Log Analysis Challenges 2
1.1.2 Importance of Machine Learning and LLMs 2
1.2 Motivation 2
1.3 Scope of the Project 3

2 PROJECT DESCRIPTION AND GOALS 4

2.1 Literature Review 4
2.2 Research Gap 11
2.3 Objectives 12
2.4 Problem Statement 12
2.5 Project Plan 12

3 TECHNICAL SPECIFICATION 14

3.1 Requirements 14
3.1.1 Functional Requirements 14
3.1.2 Non-Functional Requirements 14
3.2 Feasibility Study 15
3.3 System Specification 16
3.3.1 Hardware Specification 16
3.3.2 Software Specification 16

4 SYSTEM DESIGN **17**
4.1 System Architecture 17
4.1.1 Log Collection and Data Ingestion 17
4.1.2 Preprocessing and Normalization 18
4.1.3 Machine Learning Model Pipeline 18
4.1.4 Prompt-Only LLM Pipeline 19
4.1.5 LoRA Fine-Tuned LLM Pipeline 19
4.1.6 Evaluation and Comparison Layer 20
4.1.7 Final Results and Analysis 20
4.2 Detailed Design 21
4.2.1 Data Ingestion Layer 21
4.2.2 Processing and Management Hub 22
4.2.3 Classical Machine Learning Pipeline 22
4.2.4 LLM-Based Pipeline 23
4.2.5 Comparative Evaluation Layer 24
4.2.6 Final Output and Analysis 24

5 METHODOLOGY AND TESTING **25**
5.1 Methodology 25
5.1.1 Dataset Description 25
5.1.2 Data Preprocessing 26
5.1.3 Machine Learning Models 26
5.1.4 LLM Input Representation 26
5.1.5 Zero-Shot LLM Evaluation 27
5.1.6 LoRA Fine-Tuning 27
5.1.7 Evaluation and Comparison 27
5.1.8 System Overview 28

6 Project Implementation **29**
6.0.1 A. Data Selection and Preprocessing 29
6.0.2 B. Machine Learning Model Implementation 31
6.0.3 C. Zero-Shot LLM Evaluation 32
6.0.4 D. QLoRA Fine-Tuning and Model Selection 32

6.0.5 E. Evaluation and Dashboard Orchestration 33

7 Results and Discussion **35**

7.1 Performance Evaluation of Machine Learning Models 36

7.1.1 Random Forest Classifier 36

7.1.2 XGBoost Classifier 37

7.2 Evaluation of Unsupervised Anomaly Detection 38

7.2.1 Isolation Forest Performance 38

7.3 Feature Importance and Interpretability 39

7.4 LLM and QLoRA-Based Threat Detection Results 40

7.4.1 Zero-Shot Baseline Performance 40

7.4.2 Impact of QLoRA Fine-Tuning 40

7.4.3 Comparative Analysis 41

7.4.4 Computational Feasibility 42

7.5 Discussion 42

8 CONCLUSION AND FURTHER ENHANCEMENTS **43**

8.1 Conclusion 43

8.2 Further Enhancements 44

8.2.1 Real-Time Log Monitoring 44

8.2.2 Integration with Security Tools 44

8.2.3 Advanced LLM Fine-Tuning 45

8.2.4 Expanded Dataset Support 45

8.2.5 Explainable AI Enhancements 45

8.2.6 Deployment and Optimization 45

8.3 Final Remark 46

References 47

EXECUTIVE SUMMARY

EXECUTIVE SUMMARY

As modern cyberattacks have become more sophisticated, the inability of static methods of detecting cyber threats has become prominent. To combat this, a new framework will be developed that utilizes the strengths of both traditional machine learning and large language models to detect cyber threats in real-time.

The UNSW-NB15 dataset will be used for the framework that contains normal and cyberattack traffic. Preprocessing will occur for the dataset prior to training both machine learning and large language models. For machine learning models, Random Forest, XGBoost, and Isolation Forest will be trained to detect cyberattacks. For the large language models, features for cyberattacks will be converted to natural language prompts that will be given to models to determine the severity of the cyberattack.

The results of the machine learning models showed that the XGBoost model performed the best with an accuracy of 90.56

For the large language models, three models will be tested: Phi-4 Mini, Llama-3.2 3B and Gemma-2 2B in zero-shot and fine-tuned models. The zero-shot models exhibited limited understanding of cyberattacks with the severity classifications ranging from 32

LIST OF TABLES

2.1 Literature Review Summary 5

LIST OF FIGURES

4.1 Cybersecurity Threat Detection Architecture 17
4.2 Cybersecurity Threat Detection Architecture 21

7.1 Comparison of Performance Metrics Across Machine Learning Models 36
7.2 Accuracy Comparison of Machine Learning Models 36
7.3 ROC Curve of the Random Forest Classifier 37
7.4 ROC Curve of the XGBoost Classifier 38
7.5 Isolation Forest Anomaly Score Distribution 38
7.6 Top Feature Importances in Random Forest 39
7.7 Top Feature Importances in XGBoost 39
7.8 LoRA Fine-Tuning Progression (Cross-Entropy Loss) 41
7.9 Comparison of Zero-Shot and QLoRA-Tuned LLM Severity Accuracy . 41

LIST OF ABBREVIATIONS

AI Artificial Intelligence

ML Machine Learning

SYMBOLS AND NOTATIONS

α Weight parameter

β Bias parameter

Δ Change in value

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

In the current digital age, organizations have become reliant on networked systems, cloud platforms, and distributed infrastructures in their day-to-day operations. Huge volumes of security logs are produced by firewalls, intrusion detection systems, servers, and applications each second by these systems. These logs will carry valuable data regarding activities of the systems, user usage, and potential security risks. Nevertheless, the process of going through such huge amount of data manually is very hard, time consuming and in most cases, it results in inaccuracies.

Conventional security monitoring platforms are primarily based on pre-established rules and well-known attack signatures to detect threats. Although these systems are effective in case of known attacks, they usually fail in case of new or developing attack patterns. This limitation has resulted in the popularization of the use of the Machine Learning (ML) techniques in the automatic identification of abnormal patterns and malicious activities in the log data.

Most machine learning models are black boxes, even though they can enhance the detection accuracy. They give forecasts, but they do not make it clear why this or that log or activity was considered suspicious. This complicates the trust that security analysts have to the system and may slow down the process of making decisions in a critical incident.

With the blistering development of Artificial Intelligence (AI) and particularly Large Language Models (LLMs), it has become possible to make them more comprehensible. LLM is able to produce human readable descriptions of identified threats, enabling analysts to comprehend the problem and respond more quickly. A smarter and more feasible cybersecurity solution can be developed by combining the reasoning and explanation aspects of LLMs with the detection ability of machine learning.

1.1.1 Security Log Analysis Challenges

Security logs are often unstructured, voluminous, and generated from a variety of heterogeneous sources. This makes preprocessing and normalization difficult. In addition, distinguishing between normal and malicious behavior requires identifying subtle patterns that are often hard to detect using conventional methods.

1.1.2 Importance of Machine Learning and LLMs

Machine learning models such as Random Forest and XGBoost can effectively classify log entries based on learned patterns. However, these models often lack interpretability. LLMs, on the other hand, are capable of understanding context and generating natural language explanations. By integrating both approaches, the system can not only detect threats but also explain them in a way that is understandable to human analysts.

1.2 MOTIVATION

This project is motivated by the increasing complexity of cyber threats and the limitations of current security analysis techniques. As organizations continuously generate massive amounts of log data, the need for intelligent systems capable of both detecting and explaining threats is growing rapidly.

- **Problem Significance:** Cyber attacks are becoming increasingly sophisticated, requiring more advanced methods to identify them.
- **Practical Motivation:** Security analysts require faster and more interpretable insights for effective response.
- **Academic Motivation:** The application of ML and LLMs in cybersecurity is an emerging field of research.
- **Technical Interest:** Combining structured ML models with unstructured language models is technically interesting.
- **Aspect of Innovation:** Adding an explanation layer with LLMs makes the system more transparent and user-friendly.

1.3 SCOPE OF THE PROJECT

This project aims to design and evaluate a hybrid cybersecurity log analysis system using machine learning-based detection and LLM-based explanations.

- **Functional Scope:**
  - Processing and analyzing security log datasets.
  - Binary classification of log entries (attack vs normal).
  - Generating human-readable explanations of detected events.
- **Non-Functional Scope:**
  - Ensuring high accuracy and low false positives.
  - Improving usability and interpretability.
  - Supporting processing of large-scale data.
- **Technical Scope:**
  - Implementation using Python, Scikit-learn, and XGBoost.
  - Explanation generation using pre-trained or fine-tuned LLMs.
  - Evaluation using standard performance metrics.
- **Project Boundaries:**
  - The system currently supports only offline analysis of historical data.
  - Real-time deployment and automated response mechanisms are not included.
  - Assumes the availability of labeled data for supervised learning.
  - Does not address advanced adversarial attack scenarios.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

Analysis of cybersecurity logs has gained increased significance with the fast-paced development of networked systems and digital infrastructures. Conventional intrusion detection systems were rule-based systems, which worked well to find out known attacks but could not identify new and emerging threats. This shortcoming resulted in the use of machine learning (ML) and deep learning (DL) methods.

Machine learning models (Random Forest, Support Vector Machines and XGBoost) have been shown to perform quite well in intrusion detection with supervision. Nevertheless, these models are very reliant on labeled data and cannot be interpreted.

Unsupervised methods, such as Isolation Forest and Autoencoders, are able to identify unknown attacks, but tend to have large false positive rates.

The latest developments have brought Large Language Models (LLMs) to cybersecurity. LLMs are able to process logs and provide human-readable explanations. Nevertheless, adaptation might be required to ensure that pre-trained LLMs can be effective in domain-specific tasks like analyzing cybersecurity logs.

To manage this, recent studies consider parameter-efficient methods of fine-tuning like LoRA (Low-Rank Adaptation), that enables adaptation of LLMs to domain-specific data using a low amount of computation. This strategy enhances performance and relevance without having to re-train the entire models.

The comparative analysis of existing studies is summarized in Table 2.1.

Table 2.1: Literature Review Summary

S/N	Author(s), Year	Study Focus	Methodology	Key Findings	Contribution
1	Palma et al. (2025) et al. (2025i)	LLMs for log analysis	Comparative study	LLMs outperformed XGBoost/-LightGBM and generated explanations	Strong evidence for LLM-based detection
2	Karlsen et al. (2023) et al. (2023d)	Benchmarking LLMs	Fine-tuning 60 LMs	DistilRoBERTa achieved F1=0.998	Validates Transformer effectiveness
3	Costa Silva et al. (2025) et al. (2025a)	Intrusion log analysis	Few-shot GPT-4o-mini	Detected anomalies without training	Supports zero/few-shot feasibility
4	Daniel et al. (2025) et al. (2025b)	NIDS rule labeling	Prompting + ML	LLMs scalable for rare labels	Supports hybrid ML+LLM
5	Ali & Kostakos (2023) **?**	HuntGPT	RF + SHAP/LIME + GPT	Actionable intrusion alerts	Supports LLM explanations
6	Al-hammouri et al. (2025) hammouri et al. (2025)	Zero-day IoT threats	Signature IDS + GPT-2	Accuracy +6.3%, fewer false positives	Supports hybrid IDS
7	Kheddar (2024) Kheddar (2024)	IDS survey	Literature review	Identifies interpretability/scalability issues	Positions current work

S/N	Author(s), Year	Study Focus	Methodology	Key Findings	Contribution
8	Houssel et al. (2024) et al. (2024b)	Explainable NIDS	GPT-4/Llama3 + RAG	Good explanations, weak detection	LLM as complement
9	Entezami et al. (2025) et al. (2025c)	Hybrid anomaly detection	LLM + CNN/LSTM	95% accuracy, fewer false alarms	Strong hybrid evidence
10	Ferrag et al. (2023) et al. (2023c)	SecurityBERT	Lightweight Transformer	98.2% accuracy, fast inference	Efficient LLM-style IDS
11	Yang et al. (2025) et al. (2025j)	LLMs in NIDS	Review/framework	LLM as processor/explainer/controller	Defines LLM role
12	Nwafor et al. (2024) et al. (2024e)	IoT IDS	Fine-tuning + embeddings	Embeddings competitive	Alternative LLM usage
13	Rajathi & Rukmani (2025) Rajathi and Rukmani (2025)	Hybrid IDS	Stacking models	99–99.9% accuracy	Strong non-LLM baseline
14	Jerábek et al. (2025) et al. (2025f)	Explainable anomaly detection	Detector + LLM	Better situational awareness	LLM for explanation
15	Elouardi et al. **?**	Hybrid CNN/LLM IDS	Survey	Reviews open problems	Context for IoT IDS
16	Mahmood et al. et al. (2025h)	IoT security framework	IF + RF + GPT-4o mini	Human-readable risk scoring	Supports semantic explanations

S/N	Author(s), Year	Study Focus	Methodology	Key Findings	Contribution
17	Sajid et al. (2024) et al. (2024g)	Hybrid ML/DL IDS	XGBoost + CNN + LSTM	¿97% accuracy	Strong baseline
18	Xu et al. (2024) **?**	LLM4Security	Systematic review	Rapid growth in LLM security tasks	Broader research context
19	Houssel et al. (2025) **?**	eX-NIDS	Prompt augmenter + LLM	¿20% better explanations	Supports prompt augmentation
20	Kumar et al. (2025) et al. (2025g)	AE–ELM IDS	Autoencoder + ELM	98.52% accuracy	High-performing non-LLM reference
21	Bentéjac et al. (2019) et al. (2019a)	Comparative analysis of boosting algorithms	Empirical comparison	XGBoost ranked second in accuracy and speed	Justifies XGBoost as a strong baseline
22	Ali et al. (2023) **?**	Review of XGBoost applications	Narrative review	XGBoost is accurate, scalable, and widely applicable	Supports XGBoost selection
23	Niazkar et al. (2024) et al. (2024d)	XGBoost in scientific prediction	Systematic review	XGBoost or hybrids achieved best performance in 74% studies	Shows robustness across domains

S/N	Author(s), Year	Study Focus	Methodology	Key Findings	Contribution
24	Chang et al. (2018) et al. (2018a)	Credit risk prediction	XGBoost + under-sampling	Outperformed Logistic Regression, SOM, and SVM	Supports handling imbalanced data
25	Imani et al. (2025) et al. (2025e)	RF vs XGBoost under imbalance	Comparative study	Tuned XGBoost + SMOTE achieved highest F1	Validates imbalance-handling strategy
26	Budholiya et al. (2020) et al. (2020)	Heart disease diagnosis	Optimized XGBoost	Achieved 91.8% accuracy	Shows importance of hyperparameter tuning
27	Sheridan et al. (2016) et al. (2016)	QSAR prediction	XGBoost vs RF vs Deep Learning	XGBoost better than RF and close to deep nets	Supports efficiency-performance tradeoff
28	Mitchell & Frank (2017) Mitchell and Frank (2017)	GPU acceleration of XGBoost	CUDA-based optimization	Achieved 3–6× speedup over CPU	Supports large-scale deployment
29	Ben Jabeur et al. (2022) et al. (2022a)	Bankruptcy prediction	FS-XGBoost	AUC 0.866–0.958	Supports feature selection integration
30	Asselman et al. (2021) et al. (2021)	Student performance prediction	Ensemble comparison	XGBoost achieved highest accuracy	Demonstrates generalization

S/N	Author(s), Year	Study Focus	Methodology	Key Findings	Contribution
31	Salman et al. (2024) et al. (2024h)	Random Forest overview	Conceptual review	RF robust to missing values, imbalance, overfitting	Theoretical basis for RF baseline
32	Speiser et al. (2019) et al. (2019c)	RF feature selection	Empirical comparison	Improves error/computation trade-off	Guides feature selection strategy
33	Pious et al. (2024) et al. (2024f)	RF prediction accuracy	Survey review	RF improves prediction but interpretability challenge	Highlights strengths/limitations
34	Jaiswal & Samikannu (2017) Jaiswal and Samikannu (2017)	RF for high-dimensional data	Empirical comparison	Handles feature selection and noise effectively	Supports RF for cybersecurity data
35	Ignatenko et al. (2024) et al. (2024c)	Improved RF with entropy gains	Modified RF	Improved accuracy significantly	Shows RF can be enhanced
36	Couronné et al. (2018) et al. (2018b)	RF vs Logistic Regression	Large-scale benchmark	RF outperformed Logistic Regression on 69% datasets	Establishes RF baseline

S/N	Author(s), Year	Study Focus	Methodology	Key Findings	Contribution
37	Arora & Kaur (2020) Arora and Kaur (2020)	RF for credit risk	Bolasso + RF	Best AUC and accuracy	Supports RF + FS approach
38	Mylonas et al. (2022) et al. (2022c)	Explainable RF	LionForests method	Effective local explanations	Comparison point for LLM explanations
39	Rachwał et al. (2023) et al. (2023e)	Improved IF with SHAP	SHAP-based weighting	Improved metrics and interpretability	Supports explainable IF
40	Chabchoub et al. (2022) et al. (2022b)	IF parameter study	Parameter tuning	Faster than IF with similar accuracy	Guides tuning
41	Xu et al. (2022) **?**	Deep Isolation Forest	Random neural representations	Improved over standard IF	Supports deep-/hybrid baselines
42	Xiang et al. (2023) et al. (2023f)	OptIForest	Multi-fork trees	Better detection and efficiency	Motivates optimized IF
43	Sri Lakshmi et al. (2024) **?**	IF in SDN security	IF on SDN traffic	Showed limitations in precision/recall/F1	Justifies hybrid methods
44	Chen et al. (2023) **?**	BS-iForest	Box-plot sampling	Improved AUC	Supports robustness improvements
45	Cheng et al. (2019) et al. (2019b)	IF + LOF ensemble	Two-layer anomaly detection	Better detection, reduced complexity	Supports ensemble anomaly detection

S/N	Author(s), Year	Study Focus	Methodology	Key Findings	Contribution
46	Abbassi et al. (2024) et al. (2024a)	Real-time IF fraud detection	Spark/Kafka architecture	Achieved 99% accuracy	Supports real-time feasibility
47	Moustafa & Slay (2015) Moustafa and Slay (2015)	UNSW-NB15 dataset creation	Dataset generation	Created realistic intrusion dataset	Primary dataset source
48	Moustafa & Slay (2016) Moustafa and Slay (2016)	UNSW-NB15 vs KDD99 comparison	Statistical analysis	UNSW more realistic and balanced	Justifies choosing UNSW
49	Moustafa et al. (2017) et al. (2017)	Big data analytics for IDS	Statistical decision-making	Effective anomaly detection	Supports ML-based IDS methodology
50	Zhou et al. (2023) **?**	LIMA: Less Is More for Alignment	Supervised instruction tuning	High-quality small tuning data sufficient	Supports efficient LoRA fine-tuning

2.2 RESEARCH GAP

Despite advancements, several challenges remain:

- Lack of interpretability in ML and DL models
- Limited domain adaptation of LLMs for cybersecurity tasks
- Dependence on large labeled datasets
- High false positive rates in anomaly detection
- Poor generalization across datasets
- Limited use of parameter-efficient fine-tuning techniques such as LoRA

2.3 OBJECTIVES

- To analyze the UNSW-NB15 dataset for intrusion detection
- To implement supervised machine learning models such as Random Forest and XGBoost
- To apply unsupervised anomaly detection techniques
- To evaluate models using accuracy, precision, recall, F1-score, and ROC-AUC
- To integrate Large Language Models for explanation generation
- To fine-tune two LLMs using LoRA for domain-specific cybersecurity tasks
- To compare performance between base LLMs and LoRA-tuned models
- To improve interpretability and usability of cybersecurity systems

2.4 PROBLEM STATEMENT

The current cybersecurity systems produce large amounts of log data, and it is not feasible to analyze it manually. Although machine learning models can identify malicious activity, they are not interpretable. Pre-trained LLMs are not domain-specific but have an explanation.

The purpose of this project is to create a hybrid system with the UNSW-NB15 dataset, in which machine learning models carry out intrusion detection, and LLMs provide explanations. Besides, two LLMs will be fine-tuned using LoRA, to enhance their performance on data related to cybersecurity, allowing a more precise and context-sensitive analysis.

2.5 PROJECT PLAN

- **Phase 1:** Dataset understanding (UNSW-NB15)
- **Phase 2:** Data preprocessing and feature engineering
- **Phase 3:** ML model implementation (RF, XGBoost, Isolation Forest)

- **Phase 4:** Model evaluation and comparison
- **Phase 5:** Integration of LLMs for explanation
- **Phase 6:** LoRA fine-tuning of LLMs on log data
- **Phase 7:** Comparative analysis (base vs fine-tuned LLMs)
- **Phase 8:** Final analysis and documentation

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

3.1.1 Functional Requirements

- The system will consume and process the UNSW-NB15 cybersecurity dataset.
- The system will preprocess raw network traffic data through cleaning, encoding, and normalization.
- The system will adopt supervised machine learning models such as Random Forest and XGBoost for intrusion detection.
- The system will utilize unsupervised anomaly detection methods such as Isolation Forest.
- The system will evaluate model performance using metrics such as accuracy, precision, recall, F1-score, and ROC-AUC.
- The system will incorporate Large Language Models (LLMs) to generate human-readable explanations.
- The system will fine-tune two LLMs using LoRA on domain-specific cybersecurity knowledge.
- The system will compare the performance of base LLMs with LoRA-tuned models.

3.1.2 Non-Functional Requirements

- **Performance:** The system must be capable of processing large-scale datasets efficiently.
- **Scalability:** The system must scale with increasing data size with minimal modifications.

- **Accuracy:** The system should maintain high detection accuracy and low false positives.
- **Interpretability:** The system must provide clear and understandable explanations.
- **Reliability:** The system is expected to produce consistent and dependable results.
- **Usability:** The system should be user-friendly through commonly used tools.
- **Portability:** The system must be operable on both local systems and cloud platforms.

3.2 FEASIBILITY STUDY

Technical Feasibility: The technical feasibility of the project is confirmed because it uses commonly available tools such as Python, Scikit-learn, and pre-trained LLMs. The UNSW-NB15 dataset is publicly accessible and suitable for training intrusion detection models. LoRA enables efficient fine-tuning of LLMs without requiring extensive computational resources.

Economic Feasibility: The project does not require costly infrastructure. It can be implemented using open-source tools and can optionally be run on platforms such as Google Colab.

Operational Feasibility: The system is designed to assist security analysts by automating threat detection and providing explanations. This reduces the manual effort required to analyze large volumes of logs.

Time Feasibility: The project is achievable within the academic timeline due to a structured development plan involving preprocessing, model development, evaluation, and LLM integration.

Usage of Symbols The symbols Weight parameter (α), Bias parameter (β), and Change in value (Δ) may be used where necessary in mathematical formulations.

3.3 SYSTEM SPECIFICATION

3.3.1 Hardware Specification

- **Processor:** Intel Core i5 / AMD Ryzen 5 or above.
- **RAM:** 8 GB minimum (16 GB recommended).
- **Storage:** 50 GB or more free space.
- **Architecture Type:** 64-bit architecture.
- **GPU (Optional):** NVIDIA GPU for faster LLM fine-tuning.

3.3.2 Software Specification

- **Operating System:** Windows / Linux / macOS.
- **Programming Language:** Python 3.8 or above.
- **Development Environment:** Jupyter Notebook / Google Colab.
- **Libraries:**
  - NumPy, Pandas (data processing)
  - Scikit-learn (ML models)
  - XGBoost (boosting model)
  - Matplotlib (visualization)
  - Transformers (LLM integration)
  - PEFT / LoRA libraries (fine-tuning)
- **Tools:**
  - Git / GitHub (version control)

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

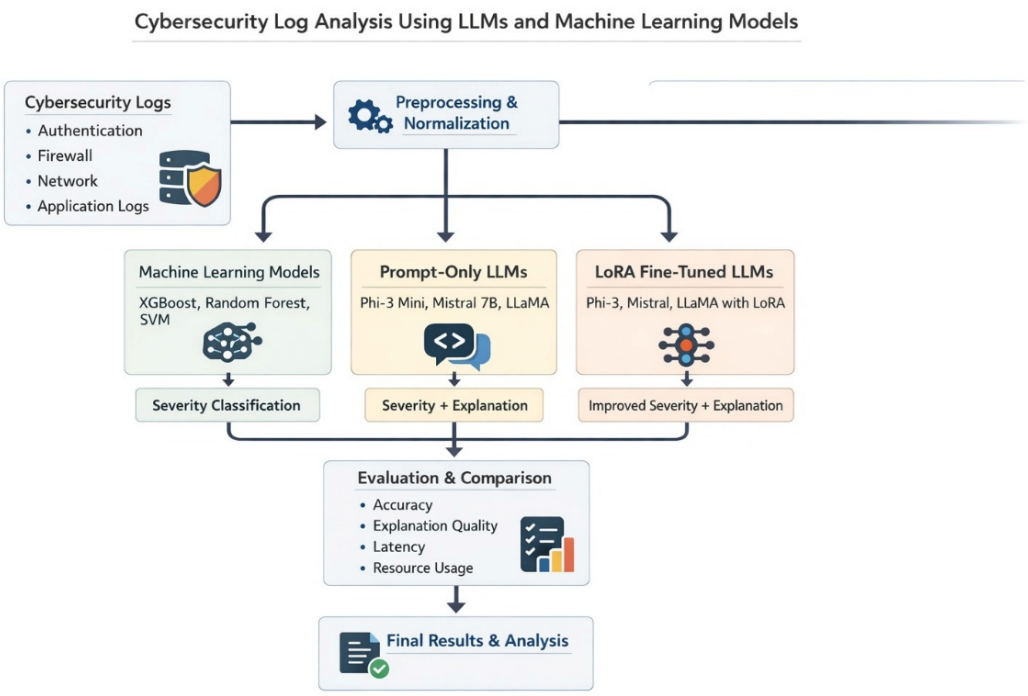

Figure 4.1: Cybersecurity Threat Detection Architecture

The proposed system architecture is designed to perform intelligent cybersecurity log analysis by integrating traditional Machine Learning (ML) models with Large Language Models (LLMs). The architecture shows automated threat detection, severity classification, explanation generation, and comparative performance evaluation.

The complete workflow of the system is illustrated in Fig. 4.2.

4.1.1 Log Collection and Data Ingestion

The first stage of the architecture involves collecting cybersecurity logs from multiple sources. These logs serve as the raw input to the system and may include:

- Authentication logs
- Firewall logs

- Network traffic logs
- Application logs

These heterogeneous logs contain both structured and semi-structured information related to network activity and security events.

4.1.2 Preprocessing and Normalization

The collected logs are passed through a preprocessing and normalization module. This module ensures that the data is clean, consistent, and suitable for downstream analysis.

The preprocessing tasks include:

- Removing redundant or irrelevant fields
- Handling missing or inconsistent values
- Encoding categorical attributes
- Normalizing numerical features
- Converting selected structured features into natural language prompts for LLM processing

This stage creates two parallel representations of the same data:

1. Structured numerical feature vectors for ML models
2. Natural language prompts for LLM-based models

4.1.3 Machine Learning Model Pipeline

The structured feature vectors are passed into the Machine Learning pipeline. This branch focuses on fast and accurate threat severity classification using traditional statistical learning methods.

The implemented ML models include:

- XGBoost
- Random Forest

- Isolation Forest

These models classify incoming logs into severity categories such as benign or malicious and provide high-performance baseline intrusion detection.

4.1.4 Prompt-Only LLM Pipeline

In parallel, the natural language representation of the logs is passed to pre-trained prompt-only LLMs.

The evaluated zero-shot LLM models include:

- Phi-4 Mini
- Llama-3.2 3B
- Gemma-2 2B

These models perform:

- Threat severity classification
- Natural language explanation generation

This pipeline demonstrates the reasoning capabilities of base LLMs without any domain-specific fine-tuning.

4.1.5 LoRA Fine-Tuned LLM Pipeline

To improve domain-specific performance, a third parallel pipeline is introduced using LoRA fine-tuned LLMs.

The fine-tuned models include:

- Phi-4 Mini with LoRA
- Llama-3.2 3B with LoRA
- Gemma-2 2B with LoRA

This branch performs:

- Improved severity classification

- Improved explanation generation

LoRA fine-tuning enables efficient adaptation of LLMs to cybersecurity-specific tasks without retraining the full model.

4.1.6 Evaluation and Comparison Layer

The outputs from all three pipelines are forwarded to a centralized evaluation and comparison module.

This layer compares the models using multiple metrics such as:

- Accuracy
- Explanation Quality
- Latency
- Resource Usage

This enables a comprehensive assessment of performance trade-offs between traditional ML and modern LLM-based approaches.

4.1.7 Final Results and Analysis

The final stage aggregates the evaluation results and produces the overall analysis.

The system identifies:

- The most accurate model
- The most resource-efficient model
- The best model for explainability
- The most balanced model for practical deployment

This architecture ensures a hybrid, scalable, and explainable cybersecurity threat detection framework capable of combining the strengths of both classical machine learning and modern large language models.

4.2 DETAILED DESIGN

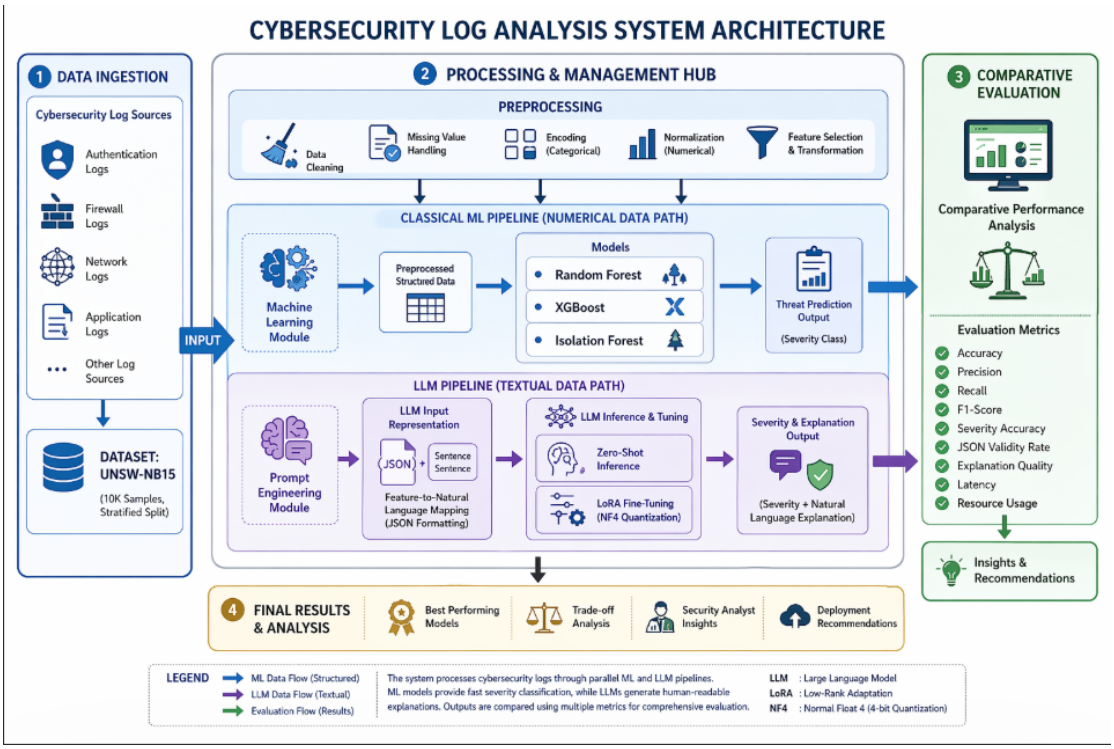

Figure 4.2: Cybersecurity Threat Detection Architecture

The diagram shows a cybersecurity threat detection system that uses both the forecasting abilities of normal Machine Learning (ML) and the thinking and explanation skills of Large Language Models (LLMs). The system is built in three main sections: bringing in the data, a Processing and Management Hub, and Comparing Results. Figure 1 shows how everything works together. The complete workflow is illustrated in Fig. 4.2.

4.2.1 Data Ingestion Layer

The Data Ingestion layer is where the system gets its information. We're using the UNSW-NB15 dataset for network traffic, and it has examples of both normal and harmful network activity, specifically Fuzzers, Exploits, DoS, Generic, Reconnaissance and Worms.

For this system, we randomly chose 10,000 of these examples, making sure to have about the same number of each type of activity. This data then goes to a section for preparing it for use.

4.2.2 Processing and Management Hub

The Processing and Management Hub is the main part of the system; it changes the data, sends it to the correct place, and makes judgements. It has two separate, at the same time operating methods: one with traditional ML and one using LLMs.

Preprocessing Module

The preprocessing module prepares raw network traffic data for downstream analysis. The following preprocessing operations are performed:

- **Data Cleaning:** Removal of duplicate, incomplete, or inconsistent records.
- **Feature Selection:** Irrelevant attributes such as `id` and `attack_cat` are removed to avoid noise and data leakage.
- **Categorical Encoding:** Features such as `proto`, `service`, and `state` are transformed using One-Hot Encoding.
- **Numerical Passthrough:** Continuous features such as byte counts and flow rates are retained in their original form.
- **Data Splitting:** The dataset is divided into training and testing sets using an 80:20 ratio with stratified sampling.

After preprocessing, the data is routed into two distinct paths.

4.2.3 Classical Machine Learning Pipeline

The numerical data path feeds structured feature vectors into the Classical ML Pipeline for binary intrusion detection.

The following models are implemented:

- **Random Forest:** An ensemble classifier using multiple decision trees to improve robustness and reduce overfitting.
- **XGBoost:** A gradient boosting algorithm that provides strong predictive performance and handles class imbalance effectively.

- **Isolation Forest:** An unsupervised anomaly detection model designed to identify unusual patterns and potential zero-day attacks.

These models produce prediction outputs indicating whether network traffic is benign or malicious.

4.2.4 LLM-Based Pipeline

The textual mapping path converts structured numerical records into natural language prompts suitable for Large Language Models.

LLM Input Representation

Selected features are transformed into descriptive textual prompts. For example:

> "Analyze this network traffic and provide a JSON-formatted threat evaluation: Network traffic using TCP protocol, service FTP, state FIN, source bytes 1234..."

The outputs are expected in a strict JSON format to ensure machine readability and easy integration into Security Operations Centers (SOCs).

LLM Inference and Tuning

The LLM pipeline consists of two modes of operation:

- **Zero-Shot Inference:** Pre-trained instruction-tuned LLMs are evaluated without domain-specific adaptation.
- **LoRA Fine-Tuning:** Parameter-Efficient Fine-Tuning is applied using Low-Rank Adaptation (LoRA) with NF4 quantization to improve domain-specific performance while minimizing computational requirements.

This pipeline produces structured outputs containing threat severity classifications and natural language explanations.

4.2.5 Comparative Evaluation Layer

The outputs from both pipelines are forwarded to the Comparative Evaluation layer for analysis.

The following performance metrics are used:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC
- JSON Validity Rate
- Severity Classification Accuracy
- Explanation Quality

This layer performs comparative performance analysis to identify the most accurate, explainable, and computationally efficient model.

4.2.6 Final Output and Analysis

The final results combine the outputs from both pipelines to provide:

- High-accuracy binary intrusion detection using ML models
- Severity classification and explanation generation using LLMs
- Comparative insights into performance, latency, and resource utilization

This hybrid architecture ensures a scalable, explainable, and efficient cybersecurity threat detection system suitable for real-world deployment.

CHAPTER 5

METHODOLOGY AND TESTING

5.1 METHODOLOGY

This study proposes a hybrid cybersecurity threat detection framework that combines traditional machine learning (ML) models with Large Language Models (LLMs). The methodology is designed to evaluate and compare three approaches: (i) classical ML-based intrusion detection, (ii) zero-shot LLM inference, and (iii) LoRA fine-tuned LLMs.

5.1.1 Dataset Description

The UNSW-NB15 dataset was used in this study for training and evaluating the proposed cybersecurity threat detection system. The dataset was generated using the IXIA PerfectStorm tool in the Cyber Range Lab at UNSW Canberra and consists of a combination of real modern network traffic and synthetic attack behaviors **?**.

The dataset contains approximately 2.54 million network flow records with 49 features representing protocol attributes, traffic statistics, and temporal characteristics. Each record is labeled as either normal or malicious, with malicious samples categorized into nine attack types: *Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode,* and *Worms*.

Dataset Usage Strategy

To effectively utilize the dataset across different modeling approaches, two distinct data usage strategies were adopted:

- **Machine Learning Models:** The full dataset was used to train and evaluate classical machine learning models, ensuring maximum exposure to data variability and improving generalization performance.

- **LLM and LoRA Fine-Tuning:** A randomly sampled subset of 10,000 records was used for LLM-based experiments. This was done due to computational con-

straints associated with fine-tuning large language models using LoRA, particularly under limited GPU memory (LiMA constraints).

The sampled subset was generated using a fixed random seed to ensure reproducibility. An 80:20 train-test split with stratified sampling was applied to maintain class distribution.

5.1.2 Data Preprocessing

The dataset was preprocessed to ensure consistency and prevent data leakage. Column names were standardized by removing leading and trailing whitespaces. The target label was separated from input features for binary classification.

Irrelevant features such as `id` were removed due to lack of predictive value. The `attack_cat` column was excluded to avoid data leakage, as it directly indicates attack types.

Categorical features (`proto`, `service`, `state`) were encoded using One-Hot Encoding with `handle_unknown="ignore"` to ensure robustness. A `ColumnTransformer` was used to apply encoding while passing numerical features without transformation.

5.1.3 Machine Learning Models

Three machine learning models were implemented as baseline classifiers:

- Random Forest
- XGBoost
- Isolation Forest (for anomaly detection)

These models were trained on structured numerical data and evaluated using standard metrics including accuracy, precision, recall, F1-score, and ROC-AUC.

5.1.4 LLM Input Representation

Since LLMs cannot directly process tabular data, selected features were converted into natural language descriptions. Six key features were chosen (`proto`, `service`, `state`, `sbytes`, `dbytes`, `rate`) to represent each network flow.

Each record was transformed into a structured sentence:

```
Network traffic using {proto}, service {service}, state {state},
source bytes {sbytes}, destination bytes {dbytes}, rate {rate}.
```

Ground truth labels were formatted as JSON objects containing `attack_type` and `severity`.

5.1.5 Zero-Shot LLM Evaluation

A zero-shot baseline was established using pre-trained instruction-tuned LLMs. Each input was provided as a prompt, and the model was instructed to output results in JSON format.

The evaluation was conducted on a fixed subset of test samples to ensure consistency. Outputs were validated and scored based on:

- Valid JSON Rate
- Attack Type Accuracy
- Severity Accuracy

5.1.6 LoRA Fine-Tuning

To improve domain-specific performance, LoRA (Low-Rank Adaptation) was applied to fine-tune two LLMs. LoRA enables efficient training by updating only low-rank adapter layers instead of full model weights.

The fine-tuning process used supervised learning with instruction-response pairs. Models were trained using 4-bit quantization (NF4) to reduce memory usage and enable training on limited hardware.

5.1.7 Evaluation and Comparison

All approaches (ML, zero-shot LLM, and LoRA-tuned LLM) were evaluated on the same test set for fair comparison.

ML models were evaluated using classification metrics, while LLMs were evaluated using structured output metrics. The results were compared to analyze:

- Detection accuracy

- Robustness of predictions
- Quality of explanations
- Impact of LoRA fine-tuning

5.1.8 System Overview

The overall system pipeline consists of:

1. Data preprocessing and feature extraction
2. ML-based threat detection
3. LLM-based explanation generation
4. LoRA fine-tuned model evaluation
5. Comparative performance analysis

CHAPTER 6

PROJECT IMPLEMENTATION

This chapter describes the practical implementation of the proposed cybersecurity threat detection framework. The system combines traditional machine learning (ML) models with Large Language Models (LLMs) to perform binary intrusion detection and generate structured threat explanations. The framework is implemented in Python using a modular architecture consisting of preprocessing, model training, inference, and evaluation pipelines. The implementation is divided into five major phases.

6.0.1 A. Data Selection and Preprocessing

The UNSW-NB15 dataset was selected for this study due to its comprehensive representation of modern network attack vectors, including *Fuzzers, DoS, Exploits, Backdoors, Generic, Reconnaissance, Shellcode, Worms,* and *Analysis*. The dataset was utilized in two ways:

- **Machine Learning Pipeline:** The full dataset was used for training and evaluating ML models.
- **LLM and LoRA Pipeline:** A randomly sampled subset of 10,000 records was used due to computational limitations associated with LLM fine-tuning.

The preprocessing pipeline (`common.py`) standardizes and transforms the raw data before model ingestion. The following steps were sequentially applied:

- **Feature Selection:** Non-informative identifiers such as `id` and multiclass indicators such as `attack_cat` were removed to prevent data leakage. The `label` column was extracted as the binary target variable.
- **Data Splitting:** The dataset was divided into 80% training and 20% testing partitions using stratified sampling to preserve the class distribution.
- **Categorical Encoding:** The categorical features `proto`, `service`, and `state` were transformed using One-Hot Encoding with `handle_unknown="ignore"`.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

def build_preprocessor(X_train):
    categorical_columns = ["proto", "service", "state"]

    return ColumnTransformer(
        transformers=[
            ("categorical",
             OneHotEncoder(handle_unknown="ignore"),
             categorical_columns),
        ],
        remainder="passthrough"
    )
```

Listing 6.1: Preprocessing Pipeline

- **Feature Passthrough:** A `ColumnTransformer` was used to encode categorical variables while allowing numerical features to pass unchanged.

For the LLM pipeline, selected structured features were converted into conversational natural language prompts. For example:

```
def convert_flow_to_prompt(row):
    input_text = (
        f"Network traffic using {row['proto']} protocol, "
        f"service {row['service']}, state {row['state']}, "
        f"source bytes {row['sbytes']}, destination bytes {row['dbytes']}, "
        f"rate {row['rate']}."
    )

    output = {
        "severity": "HIGH" if row['label'] == 1 else "LOW"
    }

    return {
        "messages": [
            {"role": "user", "content": input_text},
            {"role": "assistant", "content": json.dumps(output)}
        ]
    }
```

Listing 6.2: Conversational Preprocessing and JSON Targeting

Ground truth labels were encoded into deterministic JSON structures containing threat severity for structured evaluation.

6.0.2 B. Machine Learning Model Implementation

Three distinct machine learning models were implemented for intrusion detection.

Random Forest Classifier

The Random Forest model (`rf.py`) was implemented using 200 decision trees (`n_estimators=200`). To address class imbalance, `class_weight="balanced"` was applied. The model generates feature importance plots and ROC curves during training.

```
rf_model = RandomForestClassifier(
    n_estimators=200,
    class_weight="balanced",
    random_state=42
)

rf_model.fit(X_train, y_train)
```

Listing 6.3: Random Forest Training

XGBoost Classifier

The XGBoost model (`xg.py`) was configured with 300 boosting rounds (`n_estimators=300`), a maximum depth of 6, and a learning rate of 0.1. A dynamic `scale_pos_weight` was computed to account for class imbalance. Subsampling and column sampling were set to 0.8 to reduce overfitting.

```
xgb_model = XGBClassifier(
    n_estimators=300,
    max_depth=6,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=class_weight
)

xgb_model.fit(X_train, y_train)
```

Listing 6.4: XGBoost Model Training

Isolation Forest

The Isolation Forest model (`isof.py`) was implemented as an unsupervised anomaly detector with 200 estimators. The contamination factor was dynamically computed

from the training labels. Predictions were remapped to align with the binary classification schema.

Each trained model, preprocessing pipeline, and feature metadata were serialized as `.joblib` artifacts for inference.

```
iso_forest = IsolationForest(
    n_estimators=200,
    contamination=estimated_contamination,
    random_state=42
)

iso_forest.fit(X_train)
```

Listing 6.5: Isolation Forest Training

6.0.3 C. Zero-Shot LLM Evaluation

A zero-shot inference pipeline was implemented to evaluate the out-of-the-box reasoning capabilities of instruction-tuned LLMs. The raw natural language network flow prompts were directly passed into the models using strict prompt templates.

The baseline evaluation measured:

- Severity Accuracy
- Valid JSON Format Rate

This phase serves as the experimental control to compare against fine-tuned models.

6.0.4 D. QLoRA Fine-Tuning and Model Selection

To improve domain-specific performance, Quantized Low-Rank Adaptation (QLoRA) was applied for efficient fine-tuning of LLMs on consumer-grade hardware.

The following optimizations were used:

- **4-Bit Quantization:** Models were loaded in 4-bit NF4 precision using the `bitsandbytes` library, reducing VRAM consumption.
- **Adapter Injection:** Trainable low-rank matrices ($r = 16$) were injected into attention projection layers such as `q_proj` and `v_proj`.

- **Gradient Accumulation:** Micro-batches were accumulated over multiple steps to prevent Out-of-Memory (OOM) errors.

Three LLMs were selected for comparative analysis:

- Phi-4 Mini
- Llama-3.2 3B
- Gemma-2 2B

The models were fine-tuned using instruction-response pairs and evaluated against their zero-shot counterparts.

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4"
)

lora_config = LoraConfig(
    r=32,
    lora_alpha=64,
    target_modules="all-linear",
    lora_dropout=0.05
)
```

Listing 6.6: 4-Bit Quantization and LoRA Configuration

6.0.5 E. Evaluation and Dashboard Orchestration

A centralized evaluation framework was implemented to compare ML models, zero-shot LLMs, and LoRA-tuned LLMs.

Machine Learning Evaluation

The evaluation script (`test.py`) loads serialized artifacts and computes:

- Accuracy
- Precision
- Recall
- F1-Score

- Balanced Accuracy
- ROC-AUC

A CSV summary and comparative bar charts are generated automatically.

LLM Evaluation

For LLM-based models, exactly 200 consistent seeded test flows were evaluated.

Metrics include:

- Severity Accuracy
- Valid JSON Format Rate

The unified dashboard generates:

- Cross-Entropy Loss Curves for LoRA convergence
- Comparative plots of Zero-Shot vs LoRA accuracy

This implementation enables a complete comparative analysis of classical ML techniques versus modern LLM-based cybersecurity threat detection systems.

CHAPTER 7

RESULTS AND DISCUSSION

This chapter presents the experimental results obtained from the implementation of the proposed hybrid cybersecurity threat detection framework. The performance of traditional machine learning models, unsupervised anomaly detection techniques, and Large Language Models (LLMs) enhanced with Quantized Low-Rank Adaptation (QLoRA) is analyzed in detail.

The objective of this evaluation is to understand how effectively each approach can identify malicious network traffic, classify threat severity, and provide reliable outputs suitable for practical deployment in cybersecurity environments such as Security Operations Centers (SOCs).

The experiments were conducted using the UNSW-NB15 dataset. For the machine learning models, the held-out testing partition was used to ensure an unbiased evaluation of generalization capability. For the LLM-based framework, 200 seeded network flow samples were used to maintain consistency across all evaluated models.

The following evaluation metrics were used throughout the experiments:

- Accuracy
- Precision
- Recall
- F1-Score
- Balanced Accuracy
- ROC-AUC
- Valid JSON Format Rate
- Threat Severity Accuracy

7.1 PERFORMANCE EVALUATION OF MACHINE LEARNING MODELS

Traditional supervised machine learning algorithms were first evaluated as the baseline intrusion detection mechanisms. These models were trained using the structured numerical and categorical features extracted from the UNSW-NB15 dataset.

The overall comparison of the machine learning models is illustrated in Fig. 7.1 and Fig. 7.2.

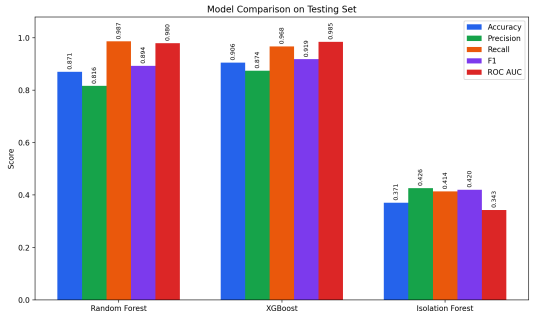

Figure 7.1: Comparison of Performance Metrics Across Machine Learning Models

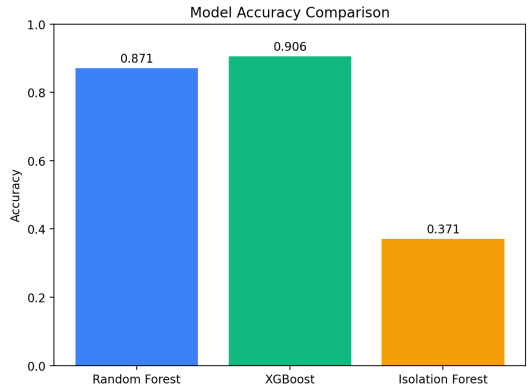

Figure 7.2: Accuracy Comparison of Machine Learning Models

7.1.1 Random Forest Classifier

The Random Forest classifier demonstrated strong intrusion detection performance, achieving an overall accuracy of **87.05%**. One of its strongest characteristics was its exceptionally high recall of **98.66%**. This indicates that the model was highly sensitive in detecting malicious traffic and produced very few false negatives.

From a cybersecurity perspective, this is particularly valuable because failing to detect an attack can be more dangerous than generating false alarms.

However, this aggressive detection behavior resulted in a slightly lower precision of **81.64%**, meaning that some benign traffic was incorrectly classified as malicious. The overall balance between precision and recall resulted in an F1-score of **89.35%**.

The model also achieved an excellent ROC-AUC score of **0.979**, indicating strong discriminative capability across multiple classification thresholds.

The ROC curve for the Random Forest model is shown in Fig. 7.3.

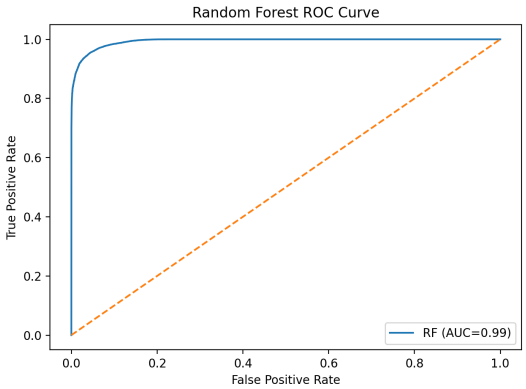

Figure 7.3: ROC Curve of the Random Forest Classifier

7.1.2 XGBoost Classifier

Among all the supervised models, XGBoost achieved the best overall performance. The model recorded an accuracy of **90.56%**, making it the most accurate machine learning classifier in this study.

Its precision of **87.44%** was significantly higher than that of the Random Forest classifier, indicating fewer false positives. Although its recall of **96.75%** was slightly lower than Random Forest, it still remained extremely strong.

This improved balance between precision and recall resulted in the highest F1-score of **91.86%**. Additionally, XGBoost achieved a Balanced Accuracy of **89.86%**, confirming its effectiveness in handling class imbalance.

The model also achieved the highest ROC-AUC score of **0.984**, suggesting superior overall discriminative power.

The ROC curve of XGBoost is shown in Fig. 7.4.

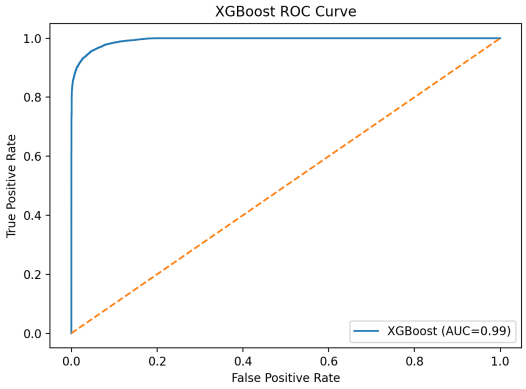

Figure 7.4: ROC Curve of the XGBoost Classifier

7.2 EVALUATION OF UNSUPERVISED ANOMALY DETECTION

The Isolation Forest model was implemented to evaluate the feasibility of anomaly detection without relying on labeled training data.

7.2.1 Isolation Forest Performance

The performance of Isolation Forest was significantly lower compared to the supervised models. It achieved an overall accuracy of only **37.06%**. Precision and recall were recorded as **42.64%** and **41.41%**, respectively, resulting in an F1-score of **42.0%**.

The ROC-AUC score was only **0.342**, indicating poor separation between normal and malicious traffic.

These results suggest that purely anomaly-based detection methods struggle to capture the complex, non-linear relationships present in modern network traffic datasets.

The anomaly score distribution generated by the model is shown in Fig. 7.5.

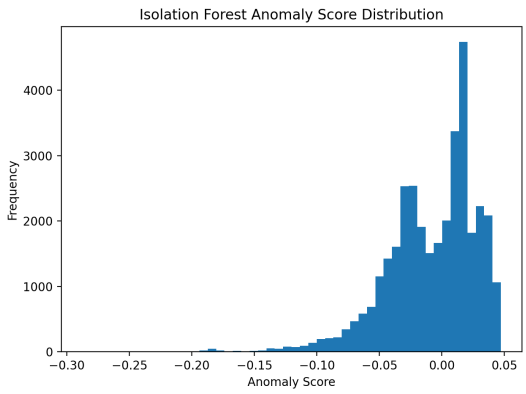

Figure 7.5: Isolation Forest Anomaly Score Distribution

7.3 FEATURE IMPORTANCE AND INTERPRETABILITY

Interpretability is an important requirement in cybersecurity applications because analysts often need to understand why a decision was made.

To improve transparency, feature importance analysis was performed on both supervised models.

For the Random Forest model, feature importance was extracted using Gini impurity reduction. For XGBoost, gain-based feature importance scores were used.

The results are shown in Fig. 7.6 and Fig. 7.7.

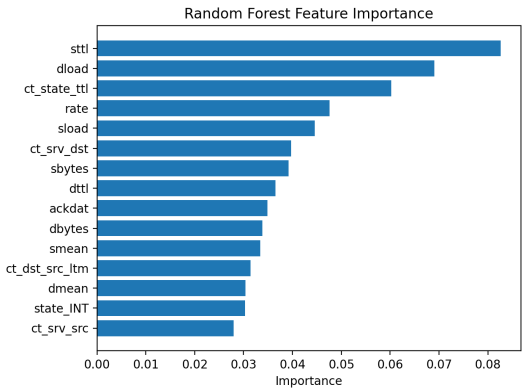

Figure 7.6: Top Feature Importances in Random Forest

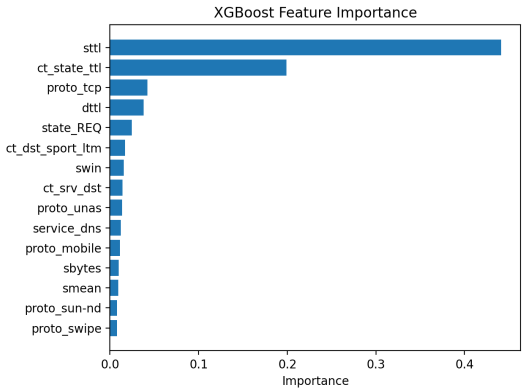

Figure 7.7: Top Feature Importances in XGBoost

Both models consistently identified connection states, byte transfer metrics, and network protocols as the most important indicators of malicious activity.

This consistency suggests that the models are learning meaningful attack-related patterns rather than overfitting to noise.

7.4 LLM AND QLORA-BASED THREAT DETECTION RESULTS

To evaluate the effectiveness of Large Language Models in cybersecurity threat analysis, three instruction-tuned models were tested in both zero-shot and QLoRA fine-tuned settings.

The evaluated models were:

- Microsoft Phi-4 Mini
- Meta Llama-3.2 3B
- Google Gemma-2 2B

The primary evaluation metrics for this stage were:

- Valid JSON Format Rate
- Threat Severity Accuracy

7.4.1 Zero-Shot Baseline Performance

The zero-shot baseline results revealed the limitations of out-of-the-box LLMs when applied to structured cybersecurity tasks.

Phi-4 Mini achieved the best zero-shot severity classification accuracy of **48.5%**. Gemma-2 followed with **45.5%**, while Llama-3.2 performed the worst at **32.0%**.

In terms of output formatting reliability, Phi-4 Mini and Gemma-2 achieved a perfect **100%** Valid JSON Rate, whereas Llama-3.2 achieved only **84.0%**, indicating occasional hallucinations or formatting inconsistencies.

These findings indicate that pre-trained LLMs possess some general reasoning capability but lack domain-specific understanding for accurate cybersecurity threat analysis.

7.4.2 Impact of QLoRA Fine-Tuning

The application of QLoRA significantly improved the performance of all models.

The most significant improvement was observed in Phi-4 Mini, where Severity Accuracy increased from **48.5%** to **92.0%**.

Similarly:

- Gemma-2 improved to **73.0%**
- Llama-3.2 improved to **74.0%**

Additionally, all fine-tuned models achieved a perfect **100% Valid JSON Rate**, completely eliminating formatting hallucinations.

The LoRA training convergence is shown in Fig. 7.8.

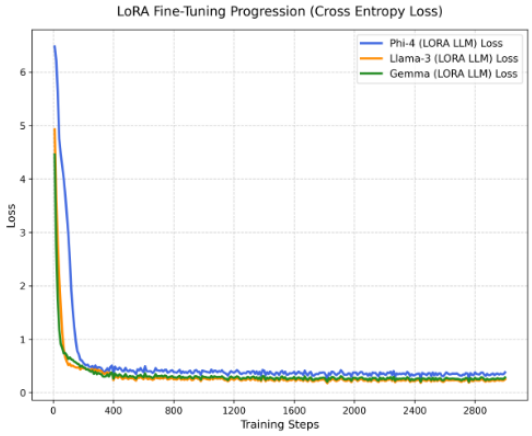

Figure 7.8: LoRA Fine-Tuning Progression (Cross-Entropy Loss)

The final severity accuracy comparison is shown in Fig. 7.9.

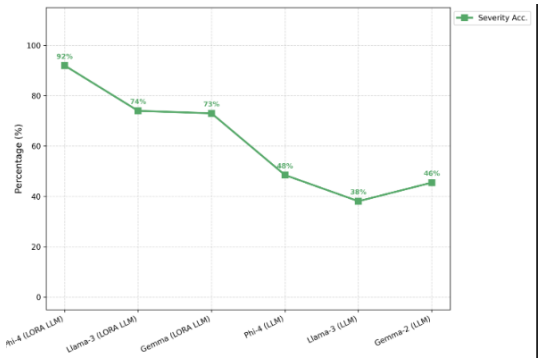

Figure 7.9: Comparison of Zero-Shot and QLoRA-Tuned LLM Severity Accuracy

7.4.3 Comparative Analysis

Among the evaluated models, Phi-4 Mini consistently outperformed the others.

This may be due to its strong reasoning-focused architecture and its pre-training on synthetic and structured tasks, which likely improved adaptation to deterministic cybersecurity workflows.

Llama-3.2 and Gemma-2, while strong general-purpose conversational models, required more adaptation to achieve competitive results.

7.4.4 Computational Feasibility

The use of 4-bit NF4 quantization and low-rank adapters ($r = 16$) made local fine-tuning possible on consumer-grade hardware.

The evaluation of 200 flows required between **250 and 538 seconds**, depending on the model.

This demonstrates the feasibility of privacy-preserving, local deployment without reliance on cloud infrastructure.

7.5 DISCUSSION

The results clearly demonstrate that supervised machine learning models outperform unsupervised anomaly detection models for binary intrusion detection on the UNSW-NB15 dataset.

Among the supervised models, XGBoost achieved the best overall balance of performance metrics, making it the strongest candidate for practical deployment.

Random Forest achieved the highest recall, making it suitable in environments where minimizing missed attacks is the top priority.

The poor performance of Isolation Forest highlights the limitations of purely anomaly-based methods.

The LLM experiments further demonstrate that zero-shot models are insufficient for domain-specific cybersecurity tasks. However, QLoRA fine-tuning dramatically improves both accuracy and reliability.

Among the evaluated LLMs, Phi-4 Mini emerged as the most effective model for severity classification and structured output generation.

Overall, the proposed hybrid framework successfully combines the high detection accuracy of traditional machine learning models with the interpretability and reasoning capabilities of fine-tuned Large Language Models.

CHAPTER 8

CONCLUSION AND FURTHER ENHANCEMENTS

8.1 CONCLUSION

This project proposed and implemented a hybrid cybersecurity threat detection framework that combines the strengths of traditional Machine Learning (ML) models with the reasoning capabilities of Large Language Models (LLMs). The objective of the system was to accurately detect malicious network activity, classify threat severity, and generate explainable outputs suitable for real-world cybersecurity environments.

The system was evaluated using the UNSW-NB15 dataset, which provides a realistic representation of modern network traffic containing both benign and malicious activities. A comprehensive preprocessing pipeline was designed to clean, transform, and prepare the data for both structured ML-based analysis and text-based LLM inference.

On the machine learning side, three models were implemented and evaluated: Random Forest, XGBoost, and Isolation Forest. Among these, XGBoost achieved the highest overall performance with an accuracy of **90.56%**, an F1-score of **91.86%**, and an ROC-AUC of **0.984**. Random Forest demonstrated exceptionally high recall, making it suitable in environments where minimizing false negatives is critical. Isolation Forest, while useful as an anomaly detection baseline, showed significantly lower performance, highlighting the limitations of purely unsupervised approaches on complex modern datasets.

On the LLM side, three instruction-tuned models—Phi-4 Mini, Llama-3.2 3B, and Gemma-2 2B—were evaluated in both zero-shot and QLoRA fine-tuned settings. The zero-shot models demonstrated limited domain-specific understanding, with severity classification accuracies ranging between **32%** and **48.5%**. However, after applying Quantized Low-Rank Adaptation (QLoRA), substantial improvements were observed across all models.

Among the evaluated LLMs, **Phi-4 Mini** achieved the best performance after fine-tuning, reaching a severity classification accuracy of **92%** while maintaining a **100% valid JSON output rate**. This demonstrates that parameter-efficient fine-tuning can significantly improve the effectiveness and reliability of LLMs in cybersecurity-specific

tasks.

The results clearly show that traditional machine learning models excel in fast and accurate binary intrusion detection, while fine-tuned LLMs provide strong reasoning, explanation generation, and structured threat analysis. By integrating both approaches, the proposed framework offers a balanced solution that is accurate, explainable, and scalable.

Overall, this project successfully demonstrates the practical feasibility of deploying a hybrid AI-driven cybersecurity threat detection system capable of supporting analysts in Security Operations Centers (SOCs) and other real-world environments.

8.2 FURTHER ENHANCEMENTS

Although the proposed system achieved promising results, several enhancements can be made in future work to improve performance, scalability, and practical usability.

8.2.1 Real-Time Log Monitoring

Currently, the system operates on offline datasets. Future work can integrate real-time log ingestion from live sources such as:

- SIEM systems
- Firewall appliances
- Intrusion Detection Systems (IDS)
- Cloud security logs

This would enable continuous real-time threat detection and alert generation.

8.2.2 Integration with Security Tools

The system can be integrated with existing cybersecurity tools and platforms such as:

- Splunk
- ELK Stack
- Wazuh

- IBM QRadar

This would improve adoption in enterprise environments.

8.2.3 Advanced LLM Fine-Tuning

Future work can explore:

- Full fine-tuning of LLMs on larger GPU clusters
- Reinforcement Learning from Human Feedback (RLHF)
- Retrieval-Augmented Generation (RAG) for contextual threat intelligence

These approaches may improve reasoning and contextual awareness.

8.2.4 Expanded Dataset Support

The system can be evaluated on additional cybersecurity datasets such as:

- CICIDS2017
- CSE-CIC-IDS2018
- NSL-KDD

This would test generalization across diverse attack environments.

8.2.5 Explainable AI Enhancements

Future work can improve interpretability by integrating:

- SHAP (SHapley Additive Explanations)
- LIME (Local Interpretable Model-Agnostic Explanations)

This would provide deeper insight into model decisions.

8.2.6 Deployment and Optimization

The system can be deployed as a web-based dashboard or cloud-native microservice architecture to improve accessibility and scalability.

Further optimization techniques such as model pruning, quantization, and ONNX acceleration can reduce inference latency.

8.3 FINAL REMARK

Cybersecurity threats continue to evolve rapidly, requiring intelligent systems that not only detect attacks accurately but also provide understandable and actionable insights.

This project demonstrates that the combination of traditional Machine Learning and modern Large Language Models can create a powerful next-generation cybersecurity analysis framework. With further refinement and real-world integration, such systems have the potential to significantly enhance automated threat detection and analyst productivity.

BIBLIOGRAPHY

Ali and Kostakos (2023). Huntgpt: An explainable ai framework for network intrusion detection using large language models.

Arora and Kaur (2020). A bolasso based random forest approach for credit risk assessment.

et al., A. (2021). A comparative study of ensemble learning methods for student performance prediction.

et al., A. (2023a). Xgboost: A review of its applications and comparative analysis.

et al., A. (2024a). Real-time fraud detection using isolation forest in distributed architectures.

et al., B. (2019a). A comparative analysis of gradient boosting algorithms.

et al., B. (2020). An optimized xgboost algorithm for the diagnosis of heart disease using bayesian tuning.

et al., B. J. (2022a). Bankruptcy prediction using a feature selection-based xgboost model.

et al., C. (2018a). A hybrid model of xgboost with under-sampling for credit risk prediction.

et al., C. (2018b). Random forest versus logistic regression: a large-scale benchmark inventory.

et al., C. (2019b). A two-layer anomaly detection ensemble using isolation forest and local outlier factor.

et al., C. (2022b). Parameter study of isolation forest for intrusion detection systems.

et al., C. (2023b). Bs-iforest: Box-plot sampling and tree selection for robust isolation forest.

et al., C. S. (2025a). Transformer-based llms in cybersecurity: An in-depth study on log anomaly detection and conversational defense mechanisms.

et al., D. (2025b). Labeling network intrusion detection system (nids) rules with mitre att&ck techniques: Machine learning vs. large language models.

et al., E. (2025c). Hybrid cnn-lstm-ae framework with llm-driven sentiment analysis for anomaly detection.

et al., E. (2026). A hybrid cnn–lstm–attention framework for intrusion detection in smart mobility networks.

et al., F. (2023c). Securitybert: A lightweight transformer-based model for cyber threat detection in iot networks.

et al., H. (2024b). Explainability of responses in gpt-4 and llama for network security.

et al., H. (2025d). ex-nids: A framework for explainable network intrusion detection leveraging large language models.

et al., I. (2024c). An improved random forest classifier using deformed entropy information gains.

et al., I. (2025e). Random forest vs xgboost: A comparative study under class imbalance using smote.

et al., J. (2025f). Explainable anomaly detection in network traffic using llm as an interpreter.

et al., K. (2023d). Benchmarking large language models for cybersecurity: A study on fine-tuning 60 language models.

et al., K. (2025g). Ae–elm ids: A novel autoencoder and extreme learning machine based intrusion detection system.

et al., M. (2017). Big data analytics for intrusion detection systems: A statistical decision-making framework.

et al., M. (2022c). Lionforests: Local interpretation of random forests for security analysis.

et al., M. (2025h). A semantic risk scoring framework for iot security using gpt-4o mini and isolation forests.

et al., N. (2024d). The application of xgboost in scientific prediction: A systematic review.

et al., N. (2024e). Fine-tuning and embedding-based large language models for iot intrusion detection.

et al., P. (2024f). Random forest for prediction accuracy: A survey and review of current trends.

et al., P. (2025i). Large language models for system log analysis: A comparative study with classical machine learning.

et al., R. (2023e). Explainable isolation forest: Shap-based feature weighting for improved anomaly detection.

et al., S. (2016). Extreme gradient boosting as a method for quantitative structure–activity relationships.

et al., S. (2019c). A comparison of random forest variable selection methods for classification and regression.

et al., S. (2024g). Hybrid machine learning and deep learning approach for robust intrusion detection systems.

et al., S. (2024h). Random forest algorithm: A comprehensive conceptual review and application guide.

et al., S. L. (2024i). Limitations of isolation forest in software defined network security.

et al., X. (2022d). Deep isolation forest for anomaly detection with random neural representations.

et al., X. (2023f). Optiforest: An optimized isolation forest using multi-fork optimal trees.

et al., X. (2024j). Llm4security: A systematic review of large language models for cybersecurity.

et al., Y. (2025j). Large language models in network intrusion detection: A systematic review and operational framework.

et al., Z. (2023g). Lima: Less is more for alignment.

hammouri et al., A. (2025). Zeroday-llm: A large language model framework for zero-day threat detection in cybersecurity.

Jaiswal and Samikannu (2017). Performance evaluation of random forest on high dimensional data.

Kheddar (2024). Intrusion detection systems based on large language models: A comprehensive survey and future directions.

Mitchell and Frank (2017). Accelerating the xgboost algorithm using gpu-based parallel computing.

Moustafa and Slay (2015). Unsw-nb15: a comprehensive data set for network intrusion detection systems.

Moustafa and Slay (2016). The evaluation of network anomaly detection systems: Statistical analysis of the unsw-nb15 dataset.

Rajathi and Rukmani (2025). A hybrid stacking-based intrusion detection system for advanced persistent threats.

Appendix A- Sample Code

```
/* C or C++ programming **/
#include <iostream>
using namespace std;
int main() {
        int a = 5, b = 10;
        cout << "Before swap: a = " << a << ", b = " << b << endl;
        int temp = a;
        a = b;
        b = temp;
        cout << "After swap: a = " << a << ", b = " << b << endl;
        return 0;
}

/* Java programming **/
public class SwapNumbers {
        public static void main(String[] args) {
                int a = 5, b = 10;
                System.out.println("Before swap: a = " + a + ", b = " + b);
                int temp = a;
                a = b;
                b = temp;
                System.out.println("After swap: a = " + a + ", b = " + b);
        }
}

/* Python programming **/
a = 5
b = 10
print(f"Before swap: a = {a}, b = {b}")
a, b = b, a  # Pythonic swap
print(f"After swap: a = {a}, b = {b}")

/* Javascript programming **/
let a = 5, b = 10;
console.log(`Before swap: a = ${a}, b = ${b}`);
let temp = a;
a = b;
b = temp;
console.log(`After swap: a = ${a}, b = ${b}`);
```

Appendix B- Publication details

- Apruzzese, G., Laskov, P., Montes de Oca, E., Mallouli, W., Brdalo Rapa, L., Grammatopoulos, A. V., & Di Franco, F. (2023). The role of machine learning in cybersecurity. *Digital Threats: Research and Practice*, 4(1), 1-38.
- Kumar, S., Gupta, U., Singh, A. K., & Singh, A. K. (2023). AI: Revolutionizing cyber security in the Digital Era. *J. Comput. Mech. Manag*, 2(3), 31-42.